

计算机网络第四章—网络层考点总结 (2024 版)

考试复习资料

2026 年 5 月 9 日

目录

1 网络层服务概述 (Network Layer Overview)	2
2 IP 地址与子网划分 (IP Addressing & Subnetting)	2
3 IP 数据报格式 (IP Datagram Format)	3
4 NAT 与 ICMP (NAT & ICMP)	5
5 路由算法 (Routing Algorithms)	5
6 拥塞控制 (Congestion Control)	9
7 服务质量 (Quality of Service, QoS)	11
8 BGP 与 SDN (BGP & SDN)	14
9 核心考点速记 (Quick Reference)	16

1 网络层服务概述 (Network Layer Overview)

网络层在协议栈中的位置

网络层位于传输层之下、数据链路层之上，负责将数据从源主机通过多跳路径传输到目的主机。
[p.3]

网络层的两大核心功能

- 1. 路由 (Routing, 控制平面): 选择数据报从源端到目的端的路径。核心是路由算法与路由协议。 [p.5]
- 2. 转发 (Forwarding, 数据平面): 将数据报从路由器的输入接口传送到正确的输出接口。 [p.5]

网络层提供的两种服务模型

- 无连接服务 (Connectionless Service): 数据报服务 (Datagram Service)。如寄信——不需要提前建立连接。每个分组独立寻路，分组携带完整的目的地址。 [p.7–9]
- 面向连接服务 (Connection-oriented Service): 虚电路服务 (Virtual Circuit Service)。如打电话——先建立逻辑连接。分组沿同一路径传输，只需携带虚电路号。 [p.10–14]

数据报服务 vs. 虚电路服务对比

[p.15]

特征	虚电路 (面向连接)	数据报 (无连接)
优点	分组携带简短的 VC 号、带宽利用率高； 连接建立后易做拥塞控制；易保证 QoS； 易实现计费	无连接开销，分组可立即发送； 对路由器故障适应性强、可及时绕道
缺点	有呼叫创建开销；路由器需存储 VC 状态； 经过失效路由器的所有 VC 都要终止	每分组携带完整目的地址（冗长）； 拥塞控制难；不易保证 QoS
实例	X.25, ATM, 帧中继, MPLS	ARPANET, Internet

虚电路交换机的工作方式

在虚电路网络中，每个节点的虚电路表记录前一节点选取的逻辑信道号和本节点选取的逻辑信道号。逻辑信道号是局部的，用于区分一条链路上的多条虚电路。 [p.12–14]

典型虚电路例子：交换虚电路 (Switched VC) 和永久虚电路 (Permanent VC)。典型实现：ATM、帧中继、X.25。 [p.12]

2 IP 地址与子网划分 (IP Addressing & Subnetting)

IPv4 地址 (IPv4 Address)

- IPv4 地址为 32 位二进制数，通常用点分十进制表示 (Dotted Decimal Notation)，如 192.168.1.1
- 每个 IPv4 地址由网络号 (Net ID) 和主机号 (Host ID) 两部分组成
- 分类编址 (Classful Addressing):

- A 类: 0.0.0.0 – 127.255.255.255, 前 8 位网络号, 首位为 0
- B 类: 128.0.0.0 – 191.255.255.255, 前 16 位网络号, 前 2 位为 10
- C 类: 192.0.0.0 – 223.255.255.255, 前 24 位网络号, 前 3 位为 110
- D 类: 224.0.0.0 – 239.255.255.255, 组播地址
- E 类: 240.0.0.0 – 255.255.255.255, 保留

子网划分 (Subnetting)

- 将一个大网络分成多个更小的子网 (Subnet)
- 子网掩码 (Subnet Mask): 32 位, 网络位部分全为 1, 主机位部分全为 0
- 通过 IP 地址 AND 子网掩码获得网络地址
- 公式:

$$\text{子网数} = 2^n \quad (\text{其中 } n \text{ 为子网位数})$$

$$\text{每个子网可用主机数} = 2^m - 2 \quad (\text{其中 } m \text{ 为剩余主机位数, 减 2 为网络号和广播地址})$$

CIDR 无分类域间路由 (Classless Inter-Domain Routing)

- 消除传统 A/B/C 类地址界限, 采用网络前缀 (Network Prefix) 表示
- 记法: IP 地址/前缀长度, 如 192.168.0.0/22
- 支持路由聚合 (Route Aggregation): 将多个连续的网络前缀聚合为一个更短的前缀, 缩小路由表
- 最长前缀匹配 (Longest Prefix Match): 查路由表时, 选择与目的地址匹配的最长前缀条目

IPv6 地址 (IPv6 Address)

- IPv6 地址为 128 位, 用冒号十六进制表示, 如 2001:0db8:85a3::8a2e:0370:7334
- 相比 IPv4 的改进: 更大的地址空间、更简洁的首部格式、内置 IPSec、更好的 QoS 支持
- IPv6 地址类型: 单播 (Unicast)、组播 (Multicast)、任播 (Anycast)
- IPv4 到 IPv6 的过渡技术: 双协议栈 (Dual Stack)、隧道技术 (Tunneling)、NAT64

3 IP 数据报格式 (IP Datagram Format)

IPv4 首部格式

IPv4 首部通常为 20 字节 (无选项字段时), 主要字段包括:

- 版本 (Version, 4 bits): IPv4 = 4
- 首部长度 (IHL, 4 bits): 以 4 字节为单位, 最小值为 5 (即 20 字节)

- 区分服务 (DS, 8 bits): 用于 QoS, 前 6 位为 DSCP
- 总长度 (Total Length, 16 bits): 首部 + 数据的总字节数, 最大 65535 字节
- 标识 (Identification, 16 bits): 数据报的标识, 用于分片重组
- 标志 (Flags, 3 bits): DF (Don't Fragment) / MF (More Fragments)
- 片偏移 (Fragment Offset, 13 bits): 以 8 字节为单位, 指示该分片在原始数据报中的位置
- 生存时间 (TTL, 8 bits): 每经过一个路由器减 1, 为 0 时丢弃
- 协议 (Protocol, 8 bits): 上层协议 (TCP=6, UDP=17, ICMP=1)
- 首部校验和 (Header Checksum, 16 bits): 仅校验 IP 首部
- 源地址 (Source Address, 32 bits)
- 目的地址 (Destination Address, 32 bits)
- 选项 (Options): 可选, 用于测试、安全等

IPv6 首部格式

IPv6 首部固定为 40 字节, 简化设计:

- 版本 (Version): IPv6 = 6
- 流标签 (Flow Label, 20 bits): 标记属于同一流的包, 用于 QoS 处理
- 有效载荷长度 (Payload Length): 不包括 40 字节首部的数据长度
- 下一个首部 (Next Header): 替代 IPv4 的 Protocol 字段, 支持扩展首部链
- 跳数限制 (Hop Limit): 替代 IPv4 的 TTL
- 源地址/目的地址 (128 bits each)
- 与 IPv4 相比: 取消了首部校验和、分片字段、选项字段 (改用扩展首部)

IP 分片与重组 (IP Fragmentation & Reassembly)

- 当数据报大于链路层 MTU 时, 路由器将数据报分片
- 分片在目的主机重组 (不在中间路由器重组)
- 一个分片丢失则整个数据报丢弃
- 片偏移以 8 字节为单位, MF=1 表示后续还有分片

4 NAT 与 ICMP (NAT & ICMP)

NAT 网络地址转换 (Network Address Translation)

- 将私有 IP 地址映射为公有 IP 地址，解决 IPv4 地址短缺问题
- 私有地址范围：
 - A 类：10.0.0.0/8
 - B 类：172.16.0.0/12
 - C 类：192.168.0.0/16
- NAT 类型：静态 NAT、动态 NAT、NAPT (端口复用，也称为 PAT)
- NAPT 使用传输层端口号区分不同内网主机，用一个公网 IP 支持多个内网主机同时上网
- NAT 的争议：打破了端到端原则，使得外部无法主动向内网发起连接

ICMP 互联网控制报文协议 (Internet Control Message Protocol)

- ICMP 是 IP 层的配套协议，封装在 IP 数据报中（协议号 =1）
- 两类 ICMP 报文：
 - 差错报告报文：目的不可达 (Type 3)、超时/“Time Exceeded” (Type 11)、参数问题 (Type 12)、源抑制 (Type 4)
 - 查询报文：回显请求/应答 (Type 8/0, 即 Ping)、路由器通告/请求、时间戳请求/应答
- **Ping**: 使用 ICMP Echo Request 和 Echo Reply 测试连通性和 RTT
- **Traceroute**: 利用 TTL 逐跳递增和 ICMP Time Exceeded 报文，追踪数据报经过的路径
- ICMP 差错报文不再产生新的 ICMP 差错报文（防止风暴）

5 路由算法 (Routing Algorithms)

路由算法概述

路由算法的目标是找到从源到目的地的最佳路径，使用汇集树 (Sink Tree)。度量指标包括：路径长度（跳数）、可靠性（出错率）、延迟时间、带宽、路由器负载、通信成本。[p.17-19]

路由算法分为两大类：[p.19]

1. 非自适应/静态路由 (Static Routing): 事先离线计算好，网络启动时下载到路由器，无法响应故障
2. 自适应/动态路由 (Dynamic Routing): 根据拓扑结构和流量的变化来改变路由选择

最优化原则 (Optimality Principle)

[p.20] 如果路由器 J 在路由器 I 到 K 的最优路由上, 那么从 J 到 K 的最优路由也会落在同一路由上。从所有源节点到一个给定目的节点的最优路由的集合形成了以目的节点为根的树, 称为**汇集树 (Sink Tree)**。路由算法的目的就是找出并使用汇集树。

最短路径路由算法 (Shortest Path Routing)

[p.22]

Dijkstra 算法

[p.23–25]

- 思想: 设 $G = (V, E)$ 是一个带权有向图, 把顶点集合 V 分成两组:
 - S : 已求出最短路径的顶点集合
 - U : 其余未确定最短路径的顶点集合
- 按最短路径长度的递增次序依次把 U 中的顶点加入 S 中, 总保持从源点 v 到 S 中各顶点的最短路径长度不大于从 v 到 U 中任何顶点的最短路径长度。
- 时间复杂度: $O(|V|^2)$ (朴素实现), $O(|E| \log |V|)$ (堆优化)
- 用于链路状态路由协议 (Link State) 计算最短路径树

洪泛算法 (Flooding)

[p.26]

- 工作原理: 将收到的每一个分组, 从除了分组到来的线路外的所有输出线路上发出
- 优点: 鲁棒性好, 总能找到最短的那条路径
- 缺点: 产生大量的重复分组
- 抑制措施:
 - 让每个分组头包含站点计数器 (跳数限制)
 - 记录下分组扩散的路径 (记录源路由器的序列号)
 - 选择性扩散 (Selective Flooding)

距离向量路由算法 (Distance Vector Routing, DV)

[p.27–35]

Bellman-Ford 方程

[p.28] 假设 $D_x(y)$ 是从 x 到 y 最小代价路径的代价值, 则:

$$D_x(y) = \min_{m \in \text{neighbors of } x} \{c(x, m) + D_m(y)\}$$

其中 m 为 x 的邻居, $c(x, m)$ 为 x 到 m 的链路代价。

核心算法

[p.27–29]

- 每个路由器维护一张表，给出到每个目的路由器的已知最短”距离”和相应输出线路
- ”距离”含义：站点数、估计时间延迟、排队的分组估计总数等
- 每个节点周期性地向邻居发送自己的距离向量
- 节点 x 收到来自邻居的新 DV 估计后，使用 B-F 方程更新自己的 DV：

$$D_x(y) \leftarrow \min_v \{c(x, v) + D_v(y)\} \quad \text{for each node } y$$

- **特点：**迭代的、分布式的。本地迭代由本地链路费用改变或邻居更新报文引起
- 所有路由器都得到正确的路由选择信息时网络进入**收敛 (Convergence)** 状态 [p.34]

无穷计算问题 (Count-to-Infinity)

[p.35]

- DV 算法在链路断开时会出现”坏消息传播慢”的问题
- 原因：节点使用邻居的路由信息计算自己的路由，形成循环依赖
- 表现为距离值不断递增直到”无穷大”
- 解决方法：
 - **水平分裂 (Split Horizon)：**不向邻居通告从该邻居学到的路由
 - **毒性反转 (Poison Reverse)：**向邻居通告从该邻居学到的路由，但距离设为无穷大
 - 定义最大距离上限（如 RIP 中的 16 跳视为不可达）

RIP (Routing Information Protocol)：基于 DV 算法的实际协议，使用跳数作为度量，最大 15 跳，每 30 秒交换路由表。

链路状态路由算法 (Link State Routing, L-S)

[p.36–42]

五步过程

[p.36–42]

1. **发现邻居节点：**路由器启动后向每个点到点线路发送 HELLO 分组，另一端的路由器发回应答说明它是谁 [p.37]
2. **测量线路开销：**发送 ECHO 分组要求对方立即响应，通过测量往返时间除以 2 得到延迟估计值。常用度量：链路带宽（反比），如 1-Gbps 以太网代价为 1，100-Mbps 以太网代价为 10。 [p.38]
3. **构造链路状态分组 (LSP)：**包含发送者标识、序列号（递增）、邻居列表及各链路开销 [p.39]

4. **发布 LSP 分组**: 使用泛洪 (**Flooding**) 将 LSP 发送给所有其他路由器。路由器记录所有收到的 (源路由器, 序列号) 对, 判断是新分组 (泛洪)、重复分组 (丢弃)、还是过时分组 (拒绝)。[p.41]
5. **计算最短路径**: 使用 Dijkstra 算法计算到每个其他路由器的最短路径 [p.42]

LSP 泛洪的关键问题

[p.40–41]

- 序列号: 32 位, 防止重复和乱序。但序列号会回转
- 路由器崩溃导致序列号丢失, 从 0 重新开始的分组会被误判为重复
- 解决: 使用年龄 (**Age**) 信息, 每秒减 1, 到 0 时丢弃该链路状态信息

OSPF (Open Shortest Path First): 基于 L-S 算法的实际协议, 使用 Dijkstra 计算最短路径, 支持多区域划分 (分层次路由)。

分层路由 (Hierarchical Routing)

[p.43]

- 对于大型网络采用”分而治之”策略
- 每个路由器只知道自己所在子网的路由信息, 而不去了解其他子网的内部结构
- 减少路由表大小和路由更新开销
- OSPF 的”区域 (Area)”概念即基于此思想

广播路由 (Broadcast Routing)

[p.44–50]

- **广播 (Broadcasting)**: 源主机同时给全部目标地址发送同一个数据包 [p.44]
- **方法 1 — 单独发送**: 给每个主机单独发送一个数据包, 效率低、浪费带宽 [p.46]
- **方法 2 — 多目标路由**: 每个分组包含一组目标, 经过路由器针对目标选路, 目标沿汇集树分散。但服务器需要知道所有的目的地地址。[p.46]
- **方法 3 — 泛洪**: 将数据包发送到所有网络节点的简单方法。无控制泛洪可能导致**广播风暴**, 环路问题严重。[p.47–48]
- **方法 4 — 生成树 (Spanning Tree)**: 源节点向所有属于该生成树的特定链路发送分组。无环路, 最佳使用带宽, 最少副本。路由器不必知道整棵树。[p.49]
- **逆向路径转发 (Reverse Path Forwarding)**: 路由器收到广播分组, 看来源那条路径是否是用来给广播源发送分组的那条线路, 是则转发到其他所有线路上, 否则丢弃。[p.50]

组播路由 (Multicast Routing)

[p.51–55]

- **组播 (Multicasting):** 源主机给网络中的一部分目标用户发送数据包 [p.51]
- **目标:** 为每个组建立组播转发树; 每个组成员只收到一个副本; 非本组成员不应收到组播分组; 路径应是最佳的 [p.52]
- **修剪生成树 (Pruning):**
 - **链路状态路由:** 每个路由器知道完整拓扑和哪些主机属于哪个组, 从每条路径末端开始逐步向根, 将不属于相应组的路由器去掉 [p.54]
 - **距离向量路由:** 使用逆向路径转发。若路由器收到组播消息但本地无组成员且无其他下游路由器有成员, 则回送 PRUNE 信息。在所有线路上收到 PRUNE 信息则继续朝源回送 PRUNE。 [p.54]
- **核心树 (Core-Based Tree, CBT):** 每个路由器只要为每个组保存一棵树 [p.54]
- **组播应用:** 音视频会议、共享电子白板、数据分发、实时数据组播、网络直播、网络游戏 [p.55]

路由算法小结

[p.56]

- **最优化原则:** 路由算法的目的是找出并使用汇集树。
- **最短路径路由:** Dijkstra 算法计算两个路由器间的最短路径。
- **洪泛算法:** 向外发送除了到来方向的所有数据包。
- **距离向量路由算法:** 根据结点间队列长度来选择路由, 最大问题是无穷计算, 水平分裂也不能完全解决。
- **链路状态路由算法:** 发现邻居 → 测量线路开销 → 封装 LSP → 发布链路状态信息 → 计算新路由。
- **分级路由:** 对大型网络分而治之, 每个路由器只知道自己所在子网的路由信息。
- **广播路由:** 单独发送/多目标/泛洪/生成树/逆向路径转发。
- **组播路由:** 建立多播转发树, 修剪生成树, CBT。

6 拥塞控制 (Congestion Control)

拥塞概述

[p.57–58]

- **拥塞 (Congestion):** 网络中数据包过多导致传输延迟或丢包, 网络吞吐量下降
- **拥塞控制 (Congestion Control):** 确保通信子网能够承载用户提交的通信量, 是一个全局性问题, 涉及主机、路由器等多种因素
- **产生原因:** 主机发送数据包数量过多超过网络承载能力; 突发流量填满路由器缓冲区造成丢包 [p.58]

拥塞控制两大策略

[p.59]

1. **开环控制 (Open-loop)**: 事先对通信流参数进行协商, 协商后参数不能动态改变。属于预防性拥塞控制, 竭力使网络总是处于无拥塞状态运行。方法包括决定什么时候接受新流量、丢弃哪些数据包。缺点: 不考虑网络的当前状态。[p.59]
2. **闭环控制 (Closed-loop)**: 根据网络状态进行动态控制, 包括反馈机制和控制机制。分为两个子类: 显式反馈 (Explicit Feedback) 与隐式反馈 (Implicit Feedback)。[p.59]

拥塞预防策略 (拥塞的网络设计策略)

[p.60]

- 数据链路层: 重传、乱序缓存、确认、流控
- 网络层: 虚电路和数据报、分组排队和服务策略、分组丢弃策略、路由算法、分组的生存时间管理
- 传输层: 重传、乱序缓存、确认、流控、超时终止

拥塞控制的途径

[p.61]

1. 提高网络供给
2. **流量感知路由 (Traffic-Aware Routing)**: 避开热门的区域, 疏散流量。计算路径权重时包含跳数、带宽、传输延迟、负载、平均排队延迟等参数。路由表反复变化可能导致路由不稳定。[p.62]
3. **准入控制 (Admission Control)**: 用于虚电路网络, 属于预防性拥塞控制。新流量将导致拥塞时不再建新的虚电路。使用漏桶 (Leaky Bucket) 或令牌桶 (Token Bucket) 描述流量, 两个参数分别给出平均速率和瞬间突发流量大小。可结合流量感知路由一起使用。[p.63]
4. **流量调节 (Traffic Throttling)** [p.64–66]:
 - 可用于数据报网络和虚电路网络
 - 检测拥塞标准: 输出线路利用率 (突发流量)、缓冲的数据包数目 (最常用)、丢包数目 (太迟)、平均分组延迟、分组延迟方差等
 - 指数加权移动平均公式:

$$u_{\text{new}} = a \cdot u_{\text{old}} + (1 - a) \cdot f$$

其中 $0 \leq a \leq 1$ 是遗忘系数, $f = 0$ 表示线路未被占用, $f = 1$ 表示线路被占用。 $a = 0$ 完全遗忘, $a = 1$ 一点不遗忘, $0 < a < 1$ 逐渐遗忘。[p.65]

- 当 $u \geq u_{\text{阈值}}$, 输出线处于警告状态, 触发动作
- **抑制包 (Choke Packet)**: 向源主机返送抑制包, 同时将拥塞数据包头加标记以防产生更多抑制包 [p.66]
- **显式拥塞通知 (ECN, Explicit Congestion Notification)**: 在 IP 头部标记拥塞位, 接收方通过 ACK 通知发送方 [p.66]

- 源节点调整负载：窗口控制（调整发送窗口大小）、速率控制（调整发送速率大小）

5. 逐跳后压 (Hop-by-hop Backpressure): 在沿途每一跳上施加反压 [p.67]

6. 负载脱落 (Load Shedding) [p.68]:

- 拥塞控制策略无法消除拥塞时采用。丢弃数据包。
- 随机早期检测 (RED, Random Early Detection, 1993 年): 当某条链路上的平均队列长度超过某个阈值时, 路由器随机丢弃部分数据包。在拥塞形成之前就开始丢包, 让 TCP 发送方提前降低速率。

7 服务质量 (Quality of Service, QoS)

QoS 概述

[p.69–70]

- 互联网本身只能提供”尽力而为的服务” (Best Effort Delivery)
- 当互联网越来越多传输多媒体信息时, 实时业务对传输延迟、延迟抖动等敏感, IP 网络不能保证 QoS 要求已成为巨大障碍
- **QoS 定义**: 网络在传输数据流时要满足一系列服务请求, 量化为带宽、时延、抖动、丢包率等性能指标 [p.69]
- 确保 QoS 需解决 4 个问题: [p.70]
 1. 应用程序需要网络提供什么样的质量?
 2. 如何规范进入网络的流量?
 3. 为了保障性能如何在路由器预留资源?
 4. 网络能否安全地接受更多流量?

QoS 度量指标 (QoS Metrics)

[p.70]

- **带宽 (Bandwidth)**: 网络路径的吞吐能力
- **时延 (Delay)**: 数据从源到目的所需时间 (包括传输时延、传播时延、排队时延、处理时延)
- **抖动 (Jitter)**: 延迟的变化/波动值。有些应用用缓存消除抖动, 但延迟问题依然存在。[p.72]
- **丢包率 (Packet Loss Rate)**: 丢失数据包所占比例

流的四个需求参数 (Flow Requirements)

[p.71] 一个从源到目标的分组流, 用 4 个基本参数描述需求:

可靠性 (Reliability), 延迟 (Delay), 抖动 (Jitter), 带宽 (Bandwidth)

流量整形 (Traffic Shaping)

[p.73–79]

漏桶算法 (Leaky Bucket Algorithm)

[p.73–75]

- 主要目的：控制数据注入到网络的速率，平滑网络上的突发流量
- 工作原理：
 - 到达的数据包被放置在底部有漏孔的桶中（数据包缓存）
 - 漏桶最多可以排队 b 个字节，受限于内存
 - 如果数据包到达时漏桶已满，丢弃数据包
 - 数据包从漏桶中以常量速率（ r 字节/秒）注入网络，平滑了突发流量

令牌桶算法 (Token Bucket Algorithm)

[p.73, 76–79]

- 漏桶不允许突发，令牌桶允许一定程度的突发数据
- 工作原理：
 - 产生令牌：周期性地以速率 r 向令牌桶中增加令牌，桶中令牌数到上限则丢弃多余令牌
 - 消耗令牌：输入数据包消耗桶中令牌。大数据包消耗更多令牌
 - 判断通过：令牌数量满足数据包需求则输出，否则丢弃
- 令牌桶公式：设桶大小为 C (字节)，令牌产生速率 ρ (字节/s)，最大传输速率 M (字节/s)
 - 无分组等待时，经过时间 $t = C/\rho$ 会发生令牌溢出
 - 以最大传输速率发送的时间： $S = C/(M - \rho)$ (在桶满且持续有数据时)

漏桶 vs. 令牌桶的区别

- 漏桶：以恒定速率输出，不允许突发（平滑作用）
- 令牌桶：允许突发数据发送（令牌积累后可以突发传输），以令牌产生速率作为长期平均速率上限
- 漏桶不缓存令牌，令牌桶缓存令牌

数据包调度 (Packet Scheduling)

[p.80–84]

- 在同一流的数据包之间以及在竞争流之间分配路由器资源的算法称为包调度算法，负责分配带宽和其他路由器资源，确定缓冲区中哪些数据包发送到输出链路上 [p.80]
- 先入先出 **FIFO** / 先来先服务 **FCFS**：采取尾丢弃 (Tail Drop)，无法满足良好的服务质量 [p.81]

- **公平排队 (Fair Queuing, Nagle 1987):** 路由器为每个输出线路使用一组单独的队列, 每个流一个队列, 线路空闲时轮流扫描队列。[p.82]
- **字节级公平排队 (Demers 1990):** 按字节进行扫描, 找到每个包结束的时刻, 按顺序排队服务。[p.83]
- **加权公平队列 (Weighted Fair Queuing, WFQ):** 给不同主机不同优先级别。公式:

$$F_i = \max(A_i, F_{i-1}) + \frac{L_i}{W}$$

其中 F_i 是第 i 个包的虚拟完成时间, A_i 是到达时间, L_i 是包长度, W 是权重。[p.84]

- **优先级调度 (Priority Scheduling):** 按优先级队列区分, 总是优先发送高优先级队列中的包

准入控制 (Admission Control)

[p.85]

- 用户在通信前先向网络提交数据流的特征和所期望的服务, 网络检查资源使用情况后作出判决
- 满足则接纳请求建立连接; 不满足则拒绝, 保证现有连接的服务质量
- **流规范 (Flow Specification):** RFC 2210/2211 精确描述可协商的参数
- 路由器将流规范转变为一组特定的资源进行预留

综合服务 (IntServ: Integrated Services)

[p.86–87]

- **特点:** 需要所有路由器在控制路径上处理每个流的消息, 维护每个流的路径状态和资源预留状态, 执行基于流的分类、调度、管理 [p.86]
- 基于资源预留协议 RSVP (Resource Reservation Protocol, RFC 2205–2210), 逐节点建立或拆除流的状态和资源预留状态 [p.87]
- **特征:** 资源预分配、全局流状态、传输控制
- **缺点:** 流状态的保留使得可扩展性差, RSVP 几乎没有具体的实现 [p.91]

区分服务 (DiffServ: Differentiated Services)

[p.89–91]

- 一种简单且可扩展的机制, 用于在 IP 网络上分类和管理网络流量并提供 QoS [p.89]
- 向语音或流媒体之类的关键网络流量提供低延迟服务, 同时向 WEB 或文件传输之类的非关键服务提供尽力而为服务
- **DSCP (DiffServ Code Point):** 在 IP 报头的 8 位 DS 字段中使用 6 位 DSCP 进行分组分类。DS 字段替换过时的 IPv4 TOS 字段。[p.89]

- **工作原理:** [p.90]
 - 边界节点根据约定的 QoS 规定, 把进入网络的流量分类成不同的流
 - 流的聚合信息用 IP 头部的 DS field 来标识
 - 内部路由节点在转发时只需根据不同的 DSCP 选择相应的调度和转发服务
- **基于类别的 QoS:** 每个类别与相应转发规则相对应, 依靠服务类型域区分。每个类别的通信流量可能先经过处理以符合特定的形状特征。可扩展性好于 IntServ。[p.91]

IntServ vs. DiffServ 对比

[p.86, 89, 91]

特征	IntServ (综合服务)	DiffServ (区分服务)
粒度	基于流 (Per-flow)	基于类别 (Per-class)
可扩展性	差 (需维护每条流状态)	好 (聚合处理)
信令协议	RSVP	无 (DS 字段标记)
资源预分配	是	否 (相对优先级)
实现复杂度	高	低

8 BGP 与 SDN (BGP & SDN)

BGP 边界网关协议 (Border Gateway Protocol)

- BGP 是互联网的核心路由协议, 属于**路径向量协议 (Path Vector Protocol)**
- 运行在 AS (Autonomous System, 自治系统) 之间, 即 **EGP (Exterior Gateway Protocol)**
- AS 内部使用 IGP (Interior Gateway Protocol), 如 RIP、OSPF
- BGP 的关键功能:
 - **AS 路径 (AS Path):** 通告路由时携带完整的 AS 跳转列表, 防止路由环路
 - **属性 (Attributes):** 如 AS-PATH、NEXT-HOP、LOCAL-PREF、MED 等
 - **选路决策:** 基于属性进行最佳路径选择 (非单纯最短路径)
 - **策略路由 (Policy-Based Routing):** 管理员可以控制允许进出 AS 的路由
- BGP 使用 TCP (端口 179) 进行可靠传输
- eBGP (外部 BGP) 和 iBGP (内部 BGP) 的区别

SDN 软件定义网络 (Software Defined Networking)

- SDN 将控制平面与数据平面分离
- 核心架构:
 - **控制层 (Control Plane):** 集中的 SDN 控制器 (如 OpenDaylight, ONOS, Ryu), 具有网络的全局视图
 - **数据层 (Data Plane):** 简单的交换机/路由器, 仅根据流表进行转发

- 南向接口 (**Southbound API**): 控制器与交换机之间的通信协议, 如 OpenFlow
- 北向接口 (**Northbound API**): 控制器与上层应用之间的接口
- SDN 的优点:
 - 集中式控制: 全局视图, 更优的流量工程
 - 可编程性: 通过软件定义网络行为
 - 开放性: 设备白盒化, 降低对厂商依赖
 - 网络虚拟化: 支持多租户隔离
- 与 SDN 相关的概念: NFV (Network Function Virtualization)

9 核心考点速记 (Quick Reference)

关键概念对照表

概念	定义/公式
数据报服务 (Datagram)	无连接, 每个分组独立路由, 携带完整目的地址 [p.7-9]
虚电路 (Virtual Circuit)	面向连接, 先建立逻辑连接, 按同一路径传输 [p.10-14]
子网掩码	$IP \wedge Mask = \text{网络地址}$
CIDR 前缀	IP 地址/前缀长度, 支持路由聚合
NAT	私有 IP $\leftrightarrow$ 公有 IP 映射; NAT 用端口号复用
Bellman-Ford 方程	$D_x(y) = \min_v \{c(x, v) + D_v(y)\}$ [p.28]
距离向量 (DV)	分布式、迭代、周期性交换 DV、收敛 [p.27-35]
链路状态 (L-S)	发现邻居 $\rightarrow$ 测量开销 $\rightarrow$ 泛洪 LSP $\rightarrow$ Dijkstra [p.36-42]
Dijkstra 算法	最短路径递增次序, 时间复杂度 $O(V ^2)$ [p.24, 42]
无穷计算	DV 链路故障时坏消息传播慢, 水平分裂/毒性反转缓解 [p.35]
洪泛 (Flooding)	每分组从所有非到来线路发出 [p.26]
广播路由	泛洪/生成树/逆向路径转发 (RPF) [p.44-50]
组播路由	建立组播树, 修剪生成树, RPF/PRUNE [p.51-55]
开环控制	预防性, 不根据当前网络状态调整 [p.59]
闭环控制	基于网络状态反馈动态调整 [p.59]
准入控制	使用漏桶/令牌桶描述流量的 (r, b) 参数 [p.63]
EWMA	$u_{\text{new}} = a u_{\text{old}} + (1 - a) f$ [p.65]
抑制包 (Choke Packet)	路由器向源主机反馈拥塞控制 [p.66]
ECN	在 IP 头标记拥塞, 由 TCP ACK 回传 [p.66]
RED	平均队列长度超阈值时随机丢弃 [p.68]
QoS 四指标	带宽、时延、抖动、丢包率 [p.70]
漏桶	恒定输出速率, 平滑流量 (不可突发) [p.75]
令牌桶	允许突发, 桶参 C , 速率 ρ , 最大 M [p.76-77]
WFQ	$F_i = \max(A_i, F_{i-1}) + L_i/W$ [p.84]
IntServ	基于流, RSVP 资源预留, 可扩展性差 [p.86-87]
DiffServ	基于类别, DSCP 标记, 可扩展性好 [p.89-91]
BGP	路径向量协议, AS 间路由, 策略选路
SDN	控制与转发分离, OpenFlow, 集中控制器

必会公式

1. **Bellman-Ford:** $D_x(y) = \min_{v \in N(x)} \{c(x, v) + D_v(y)\}$ [p.28]
2. 子网主机数: $2^m - 2$ (m 为剩余主机位数)
3. 子网数: 2^n (n 为子网位数)
4. **EWMA 拥塞检测:** $u_{\text{new}} = a \cdot u_{\text{old}} + (1 - a) \cdot f, 0 \leq a \leq 1$ [p.65]
5. 令牌桶溢出时间: $t = C/\rho$ [p.77]

6. 令牌桶最大突发传输时间: $S = C/(M - \rho)$ [p.77]

7. WFQ 虚拟完成时间: $F_i = \max(A_i, F_{i-1}) + L_i/W$ [p.84]

协议与端口/号速查

协议	类型	备注
RIP	DV 路由	基于 UDP 520, 最大 15 跳
OSPF	L-S 路由	基于 IP 协议号 89, Dijkstra 算法
BGP	路径向量路由	基于 TCP 179, AS 间
ICMP	控制协议	IP 协议号 1
RSVP	资源预留	IntServ 信令, RFC 2205